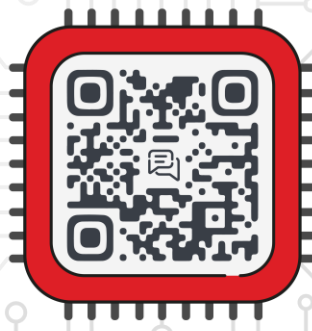




2

7

840 sp

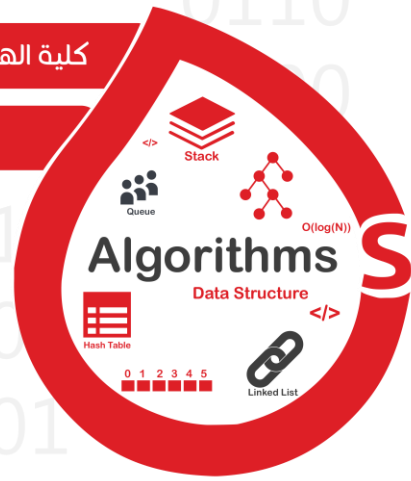
عملي

كلية الهندسة المعلوماتية

السنة الثانية

التعقيد الزمني

عملي مشترك



خوارزميات

RB Informatics; 27/10/2025

قواعد لكتابة التعقيد:

- يُهمَل الثَّابِت عند حساب التعقيد، مثال:

$$O(C.n) \Rightarrow O(n)$$

$$O(C + n) \Rightarrow O(n)$$

- عندما تكون n كبيرة، فإنَّ الحدَّ ذو الأسَّ الأكبر هو الَّذي يهيمن على نمو التعقيد، أمَّا الحدود ذات الأسَّ الأصغر تهمل أمامه، مثال:

$$O(n^2 + n) \Rightarrow O(n^2)$$

$$O(n^x + n^y) = O(n^{\max(x,y)})$$

أنواع التعقيد الزمني:

1. $O(1)$ ، عدد خطوات الخوارزمية ثابت لا يتأثر بالدخَل (كلّ تعليمة تنفَّذ مرّة واحدة).

المثال	الشرح
<code>int fun() {</code>	تعريف الدالة لا يُحسب خطوة
<code>int s, b;</code>	تعريف المتحوّلات لا يُحسب خطوة
<code>s = s + b;</code>	إسناد قيمة للمتغير يُحسب خطوة عملية الجمع خطوة
<code>return s;</code> <code>}</code>	إرجاع قيمة يُحسب خطوة

Time complexity = 2 + 1 = 3

$$O(3) \Rightarrow O(1)$$

مجموع العمليات 3، و هي عمليات غير مكلفة للوقت (عمليات ذات زمن تنفيذ ثابت) لا تحسب في معادلة Big-O.

2. $O(n)$ ، تنفذ العمليات n مرة كما هو الحال في الحلقات:

المثال	الشرح
<pre>int fun (int n) {</pre>	
<pre> int s = 0;</pre>	إسناد
<pre> for(int i = 1; i<=n; i++)</pre>	عملية جمع و إسناد تتكرر n مرة $\Rightarrow 2n$
<pre> s += i;</pre>	
<pre> return s;</pre>	عملية إرجاع
<pre>}</pre>	

Time complexity = $1 + 2n + 1 = 2n + 2 \approx O(n)$

تمرين:

<pre>int n = 10;</pre>	n عدد ثابت لا يتعلق بالدخل
<pre>for (int i = 0; i < 10; i++)</pre>	$O(10) \Rightarrow O(1)$
<pre> System.out.println(i);</pre>	

تمرين:

<pre>void fun(int n){</pre>	
<pre> for (int i = 0; i < n; ++i)</pre>	تتعلق بالدخل
<pre> System.out.println("Smile");</pre>	سيتم تنفيذ تعليمة الطباعة n مرة $\Rightarrow O(n)$
<pre>}</pre>	

3. $O(\log_x n)$ ، عندما تتغير قيمة عدّد الحلقة بشكل مضاعف أو مقسوم.

التوضيح:

لتكن لدينا الحلقة:

```
for (int i = 1; i <= n; i = i*2)
```

$i=1$	2^0	عند الدخول للحلقة أول مرة
$i=2$	2^1	عند الدخول للحلقة للمرة الثانية
$i=4$	2^2	عند الدخول للحلقة للمرة الثالثة
$i=n$	2^k	عند الدخول للحلقة للمرة n

$$2^k = n$$

نأخذ $\log$ للطرفين:

$$\log 2^k = \log n$$

$$k \log 2 = \log n$$

$$\log 2 \approx 1$$

$$k = \log n$$

تمرين:

```
int fun(int n){
    int sum = 0;
    int i = 1;
    while(i < n){
        i *= 3;
        sum += i;
    }
```

(1) يهمل
(1) يهمل
 $O(\log_3 n)$

Time Complexity:

$$O(\log_3 n)$$

4. $O(n^k)$ ، عندما يحوي الكود حلقتين أو أكثر بشكل متداخل حيث k عدد الحلقات.

ملاحظة:

في حال كانت الحلقات منفصلة (غير متداخلة) نجمع تعقيد الحلقات.

مثال:

```
for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
        sum += j;
```

هنا لدينا حلقتان متداخلتان؛ فالتعقيد سيكون $n \times n$ ،

$$\text{أي } O(n^2).$$

أما لو كانت:

```
for(int i = 0; i < n; i++)
    sum += i;
for(int j = 0; j < n; j++)
    sum += j;
```

هنا لدينا حلقتان منفصلتان تعقيد كل منهما n ،

$$\text{أي } O(2n) \approx O(n).$$

ماذا لو كانت الحلقة بهذا الشكل ؟

```
for(int i = 0; i < n; i++)
    for(int j = 0; j < i; j++)
        sum += j;
```

في هذه الحالة:

عدد العمليات = j مرة لكل i ، و هي متتالية حسابية.

توضيح شكل المتتالية الحسابية:

$i \quad i \quad i$

$$S = 0 + 1 + 2 + \dots + (n-1) = \frac{n}{2}(0 + (n-1)) = \frac{n(n-1)}{2}$$

$$\frac{n}{2}(a_0 + a_n)$$

و عدد مرات تكرار الدخول للحلقة الداخلية يساوي i في كل مرة ندخل الحلقة.

$$\sum_{i=0}^n i = \frac{n(n-1)}{2} = \frac{n^2+n}{2} \Rightarrow O(n^2)$$

مثال:

(2) $A = B * 3;$

عملية حسابية و إسناد

(n) $\text{for}(\text{int } i = 0; i < n; i++)$

$C = A * B;$

بالشرط: إما سينفذ if أو else

$\text{if}(\text{شرط}) \{$

نأخذ الحالة الأسوأ:

(4) $A = A * C;$

تنفيذ if:

$B = C * B;$

يوجد عمليتان حسابيتان بالإضافة إلى إسناد

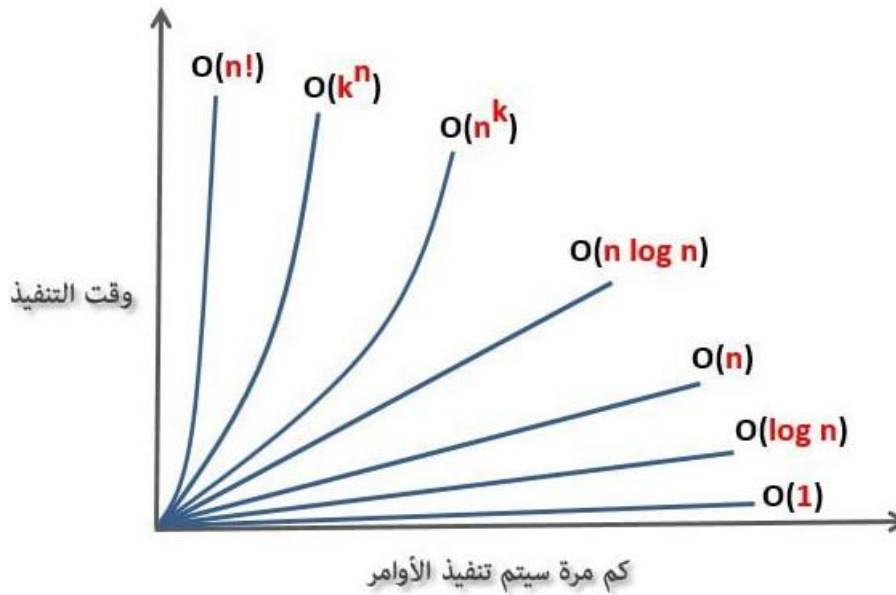
else

(2) $B = A * C;$

$$\text{Time complexity} = 2 + n + 4 \approx O(n)$$

تراتبية صفوف التعقيد من الأفضل إلى الأسوأ (Big O Complexity)

من كلمة Order بمعنى أن هناك ترتيب متزايد في زمن الخوارزمية مع زيادة حجم المدخلات.



عندما يكون حجم البيانات (n) صغيراً، فإن فروقات الأداء بين الخوارزميات تكون ضئيلة، و عندما يتصاعد حجم البيانات (n) تكون فروقات الأداء أوضح.

ملاحظة: التعقيد $O(\sqrt{n})$ أفضل من $O(n)$ و أسوأ من $O(\log n)$.

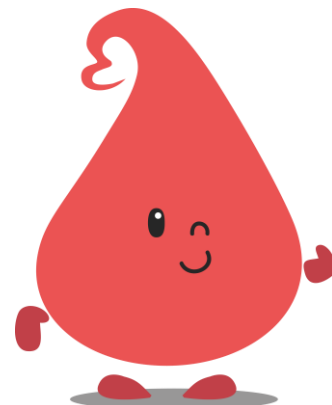
البحث عن عنصر في المصفوفة

1. البحث الخطّي:

نمرّ على جميع عناصر المصفوفة عنصراً تلو الآخر:

- الحالة الأسوأ: $O(n)$ أن يكون هذا العنصر موجود في آخر index في المصفوفة.
- الحالة الأفضل: $O(1)$ أن يكون هذا العنصر موجود في أول index.

```
1 for(int i = 0; i < n; i++) {
2     if(key == arr[i])
3         System.out.println(i);
4     return -1;
```



البحث في مصفوفة **مرتبة**، تبدأ بفحص العنصر الواقع في المنتصف و تقارنه مع العنصر المطلوب (key):

- الحالة الأفضل: أن يكون العنصر موجود في المنتصف.
 - إذا كان key أكبر يتمّ البحث في النصف الأكبر من المصفوفة.
 - إذا كان key أصغر يتمّ البحث في النصف الأصغر من المصفوفة.
- و تتكرّر العملية حتى يتمّ العثور على العنصر المطلوب:

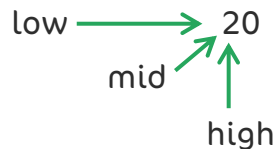


$5 < 20$ نبحث في القسم الأكبر:



$12 < 20$

$20 = 20$



حجم المصفوفة هنا 7.

استطعنا إيجاد key بـ 3 خطوات، و سيكون فرق الأداء أوضح و أفضل من أجل n كبير.

الخوارزمية: مصفوفة مرتبة تصاعدياً:

```
1 int binarySearch(int arr[], int size, int key){
2     int low = 0;
3     int high = size - 1;
4     while(low <= high){
5         int mid = (low + high)/2;
6         if(arr[mid] == key)
7             return mid;
8         else if(arr[mid] < key)
9             low = mid + 1;
10        else
11            high = mid - 1;
12    }
13    return -1;
14 }
```



شرح تعقيد هذه الخوارزمية:

رقم المرور	طول المصفوفة
1	n
2	$\frac{n}{2}$
3	$\frac{n}{4} = \frac{n}{2^2}$
$\vdots$	$\vdots$
$\vdots$	$\vdots$
k	$\frac{n}{2^{k-1}}$

$$\frac{n}{2^{k-1}} = 1$$

$$n = 2^{k-1}$$

$$\log n = (k - 1) \log 2$$

$$\log 2 \approx 1$$

$$\log n = k - 1$$

$$k = \log n + 1$$

تھمل

أسوأ حالة

$$k = \log n$$

Linear Search $O(n)$ Binary Search $O(\log n)$

Keep Working Hard
Don't Lose Hope... Try again

